AI-NIDS: Installation & Operation Manual

Complete Setup, Deployment & Execution Guide

---

1. System Requirements & Prerequisites

Hardware Requirements

- Single-PC Mode: 1 Computer (Windows 10/11, macOS, or Linux) with at least 8 GB RAM and 2 CPU cores.
- Two-PC Cyber Range Mode: 2 Computers connected to the same local Wi-Fi or Ethernet subnet (e.g., 10.64.157.0/24).

Software Prerequisites

- Node.js: Version v18.0.0 or higher (Download Node.js (https://nodejs.org/))
- Python: Version 3.10 or higher (Download Python (https://www.python.org/))
- Package Managers: npm (bundled with Node.js) and pip (bundled with Python)

---

2. Installation & Dependency Setup

Step 1: Clone or Open the Repository

```
git clone https://github.com/your-org/AI_NIDS.git
cd AI_NIDS
```

Step 2: Install Node.js Frontend Dependencies

```
npm install
```

Step 3: Install Python Backend Dependencies

```
pip install fastapi uvicorn pydantic psutil httpx
```

---

3. Configuration Setup (.env)

The application automatically reads configuration from .env in the project root.

A. Configuration for PC 2 (NIDS / SOC Server — 10.64.157.145)

Create or edit .env on PC 2:

```
# Client Role &amp;amp; Operation Mode
VITE_CLIENT_ROLE=NIDS_SOC
VITE_MODE=LIVE

# NIDS Backend Connectivity
VITE_NIDS_HOST=10.64.157.145
VITE_NIDS_PORT=8000
VITE_NIDS_WS_PORT=8000

# Attacker &amp;amp; Lab Victim Target
VITE_ATTACKER_HOST=10.64.157.140
VITE_LAB_TARGET_HOST=10.64.157.145

# Backend Server Bindings
LAB_SENSOR_HOST=10.64.157.145
LAB_ATTACKER_HOST=10.64.157.140
LAB_TARGET_HOST=10.64.157.145
LAB_SUBNET=10.64.157.0/24
```

B. Configuration for PC 1 (Attack Lab Controller — 10.64.157.140)

Create or edit .env on PC 1:

```
# Client Role &amp;amp; Operation Mode
VITE_CLIENT_ROLE=ATTACK_LAB
VITE_MODE=LIVE
```

```
# Target NIDS Server (PC 2)
VITE_NIDS_HOST=10.64.157.145
VITE_NIDS_PORT=8000
VITE_NIDS_WS_PORT=8000

# Attacker &amp;amp; Target IPs
VITE_ATTACKER_HOST=10.64.157.140
VITE_LAB_TARGET_HOST=10.64.157.145
```

---

4. Windows Firewall Setup (For Two-PC Wi-Fi Testing)

If Windows Firewall is enabled on either PC, inbound connections on ports 8000 and 5173 must be permitted for the private local network.

On PC 2 (Run in PowerShell as Administrator):

```
# Allow inbound FastAPI on port 8000
netsh advfirewall firewall add rule name="AI-NIDS FastAPI Backend (8000)" dir=in action=allow protocol=TCP localport=8000 profi

# Allow inbound Vite Frontend on port 5173
netsh advfirewall firewall add rule name="AI-NIDS Vite Frontend (5173)" dir=in action=allow protocol=TCP localport=5173 profile
```

On PC 1 (Run in PowerShell as Administrator):

```
# Allow inbound Vite Frontend on port 5173
netsh advfirewall firewall add rule name="AI-NIDS Vite Frontend (5173)" dir=in action=allow protocol=TCP localport=5173 profile
```

---

5. How to Run the Project

Option A: Two-PC Physical Cyber Range Deployment

Step 1: Start Services on PC 2 (10.64.157.145)

Open two terminals on PC 2:

- Terminal 1: Start FastAPI Backend Server

```
python backend/main.py
```

(Server listens on 0.0.0.0:8000)

- Terminal 2: Start SOC Frontend Dashboard

```
npm run dev
```

(Vite listens on 0.0.0.0:5173)

- Open SOC Dashboard in PC 2 Browser:

■ http://10.64.157.145:5173/?role=NIDS_SOC (http://10.64.157.145:5173/?role=NIDS_SOC)

Step 2: Start Services on PC 1 (10.64.157.140)

Open terminal on PC 1:

- Start Attack Lab Frontend Console

```
npm run dev
```

(Vite listens on 0.0.0.0:5173)

- Open Attack Lab in PC 1 Browser:

■ http://10.64.157.140:5173/?role=ATTACK_LAB (http://10.64.157.140:5173/?role=ATTACK_LAB)

Option B: Single-PC Standalone / Unified Mode

If running on a single laptop/workstation for development or demonstration:

- Start FastAPI Backend:

```
python backend/main.py
```

- Start Frontend:

```
npm run dev
```

• Open Unified Dual Dashboard:

■ http://localhost:5173/ (http://localhost:5173/)

---

6. Verifying the End-to-End Event Lifecycle

Once both machines are running, follow this step-by-step test procedure:

• Check Status on PC 1 & PC 2:

• PC 1 top diagnostic strip displays: ROLE: ATTACK_LAB, API: CONNECTED, WEBSOCKET: CONNECTED.

• PC 2 top diagnostic strip displays: ROLE: NIDS_SOC, API: CONNECTED, WEBSOCKET: CONNECTED.

• Launch Attack from PC 1:

• In PC 1 browser, navigate to the Attack Lab tab.

• Click START TEST on any scenario (e.g. DOS_SYN_FLOOD [HIGH]).

• Observe Live Reaction on PC 2:

• PC 2 immediately pops a Toast Notification: ● LAB ATTACK STARTED: DOS_SYN_FLOOD (HIGH) from 10.64.157.140 against 10.64.157.145.

• The topbar Notification Bell counter increments (■ 1).

• The Lab Activity banner switches to LAB TEST ACTIVE.

• Wait for Scenario Timeout (e.g. 20s):

• PC 2 displays completion toast: ● LAB ATTACK COMPLETED.

• The active banner returns to STANDBY / IDLE.

• The completed session appears permanently in RECENT LAB ACTIVITY & ATTACK HISTORY with status COMPLETED.

• Verify Page Refresh Resilience:

• Press F5 (Refresh) in PC 2's browser.

• The completed attack record and notification history remain fully intact via GET /api/lab/history and GET /api/notifications.

---

7. Troubleshooting & Frequently Asked Questions (FAQ)

Q1: ERR_CONNECTION_TIMED_OUT when loading http://10.64.157.140:5173

• Cause: Windows Defender Firewall on PC 1 is blocking incoming TCP traffic on port 5173.

• Fix: Run the PowerShell firewall command in Section 4 on PC 1, or verify both machines are connected to the same Wi-Fi network profile.

Q2: Port 5173 is already in use (Port 5173 is in use, trying another one...)

• Cause: An orphan Node.js process is already occupying port 5173.

• Fix: Run the following command in PowerShell to clear lingering processes:

```
Get-Process -Name node -ErrorAction SilentlyContinue | Stop-Process -Force
```

Then run npm run dev.

Q3: PC 1 shows API: DISCONNECTED or WEBSOCKET: DISCONNECTED

• Cause: PC 1 cannot reach PC 2's backend on 10.64.157.145:8000.

• Fix:

• Confirm FastAPI is running on PC 2 (python backend/main.py).

• Verify network connectivity from PC 1 using PowerShell:

```
Test-NetConnection 10.64.157.145 -Port 8000
```

• Ensure PC 2's Windows Firewall allows port 8000 (Section 4).

Q4: Source IP shows as 10.64.157.145 instead of 10.64.157.140

• Cause: The browser on PC 1 is accessing PC 2's frontend directly instead of running its own local frontend.

• Fix: Run npm run dev locally on PC 1 and open http://10.64.157.140:5173/?role=ATTACK_LAB.